



# ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR

Membre de 

---

HONORIS UNITED UNIVERSITIES

## Rapport de Projet DJANGO

### SondagePro – Application Web de Sondages Dynamiques

#### Réalisé par:

Mouad Mezyan  
Marouane Younsi

#### Établissement:

EMSI

#### Année Universitaire:

2025-2026

#### Encadré par:

M.ABOUABID Hamza

---

### Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet. Nos remerciements s'adressent en premier lieu à nos professeurs pour leur encadrement précieux, leurs conseils avisés et leur disponibilité constante. Leur expertise a été un pilier essentiel dans la structuration de notre démarche et la résolution des défis techniques rencontrés.

Nous remercions également nos familles et amis pour leur soutien inconditionnel et leur patience tout au long de ce parcours. Leur encouragement a été une source de motivation inestimable.

Enfin, nous souhaitons remercier l'ensemble de la communauté open-source, dont les ressources et les outils (Django, Python, MySQL, Bootstrap, Chart.js) ont été fondamentaux

pour le développement de SondagePro. Leur contribution collective a permis de bâtir une base solide pour notre application.

---

---

## Table des matières

|                                          |    |
|------------------------------------------|----|
| Remerciements .....                      | 1  |
| Table des matières .....                 | 2  |
| Résumé du projet.....                    | 4  |
| Introduction générale.....               | 5  |
| 1. Présentation du projet.....           | 5  |
| Contexte.....                            | 5  |
| Objectifs.....                           | 5  |
| Problématique.....                       | 6  |
| Solution proposée.....                   | 6  |
| 2. Cahier des charges.....               | 6  |
| Besoins fonctionnels .....               | 7  |
| Besoins non fonctionnels.....            | 8  |
| Contraintes techniques .....             | 9  |
| Technologies choisies.....               | 9  |
| 3. Analyse et conception.....            | 10 |
| Architecture MVT Django .....            | 10 |
| Description de la base de données .....  | 11 |
| Diagramme de cas d'utilisation.....      | 11 |
| Diagramme de séquence .....              | 13 |
| Diagramme de classes .....               | 14 |
| Diagramme de transition d'état.....      | 17 |
| Diagramme d'activité.....                | 19 |
| 4. Conception de la base de données..... | 20 |
| Entités de la base de données .....      | 20 |
| 5. Développement de l'application .....  | 22 |
| Structure du projet Django.....          | 22 |
| Gestion des routes (URLs) .....          | 23 |
| Gestion des vues .....                   | 24 |

|                                                             |    |
|-------------------------------------------------------------|----|
| Modèles Django.....                                         | 25 |
| Formulaires dynamiques (Django Forms) .....                 | 26 |
| Authentification (Django Authentication) .....              | 26 |
| Sécurité CSRF .....                                         | 26 |
| Responsive Design Bootstrap .....                           | 26 |
| JavaScript AJAX / Fetch API .....                           | 26 |
| Intégration Chart.js .....                                  | 27 |
| 6. Fonctionnalités implémentées .....                       | 27 |
| Authentification et Gestion des Utilisateurs .....          | 27 |
| Création de Sondages Dynamiques.....                        | 27 |
| Questions Conditionnelles.....                              | 27 |
| Partage via Lien Unique .....                               | 27 |
| Exportation des Résultats (CSV).....                        | 28 |
| Tableau de Bord et Statistiques .....                       | 28 |
| Visualisation Graphique avec Chart.js .....                 | 28 |
| Notifications .....                                         | 28 |
| Protection par Mot de Passe.....                            | 28 |
| Limitation par IP .....                                     | 28 |
| 7. Difficultés rencontrées et solutions .....               | 28 |
| Compatibilité MySQL .....                                   | 29 |
| Communication Frontend/Backend (AJAX/Fetch API) .....       | 29 |
| Gestion des relations ORM (One-to-Many, Many-to-Many) ..... | 29 |
| Validation des formulaires dynamiques.....                  | 29 |
| Sécurité (CSRF, XSS, Injection SQL).....                    | 30 |
| Gestion des permissions et accès .....                      | 30 |
| 8. Tests et validation .....                                | 30 |
| Tests fonctionnels.....                                     | 31 |
| Validation des formulaires .....                            | 31 |
| Tests utilisateurs (User Acceptance Testing - UAT) .....    | 31 |
| Captures des résultats .....                                | 32 |
| 9. Perspectives d'amélioration .....                        | 34 |
| Notifications temps réel .....                              | 34 |
| API mobile .....                                            | 34 |
| IA pour analyse des réponses .....                          | 34 |

|                                                |    |
|------------------------------------------------|----|
| Système de recommandation.....                 | 34 |
| Multilingue.....                               | 34 |
| Hébergement cloud.....                         | 35 |
| 10. Conclusion générale.....                   | 35 |
| 11. Webographie.....                           | 35 |
| 11. Annexes.....                               | 36 |
| Extraits de code .....                         | 36 |
| Captures d'écran .....                         | 36 |
| Structure du projet.....                       | 41 |
| Commandes d'installation et de démarrage ..... | 42 |

---

---

## Résumé du projet

Le projet **SondagePro – Application Web de Sondages Dynamiques** a pour objectif de concevoir et de développer une plateforme web robuste et intuitive permettant la création, la diffusion, la gestion et l'analyse de sondages en ligne. Face à un besoin croissant d'outils flexibles pour la collecte de données et l'évaluation d'opinions, SondagePro propose une solution complète, allant de la conception de questionnaires personnalisables à l'analyse graphique des réponses et à l'exportation des données.

L'application est développée en utilisant le framework web **Django** (Python) suivant l'architecture Model-View-Template (MVT), avec **MySQL** comme système de gestion de base de données. L'interface utilisateur est construite avec **Bootstrap 5** pour un design réactif et moderne, tandis que **JavaScript (ES6)** et **Chart.js** sont employés pour les fonctionnalités dynamiques et la visualisation des données. Le projet intègre des fonctionnalités avancées telles que la gestion des utilisateurs, les questions conditionnelles, les sondages anonymes, les notifications et la possibilité de créer des modèles de sondages réutilisables.

Ce rapport détaille les différentes phases du projet, depuis l'analyse des besoins jusqu'au développement et aux perspectives d'amélioration. Il met en lumière les choix technologiques, la conception architecturale, les défis rencontrés et les solutions apportées, ainsi que les fonctionnalités implémentées et les tests effectués. L'objectif est de présenter une application fonctionnelle, sécurisée et évolutive, répondant aux exigences d'un environnement universitaire et professionnel.

---

---

## Introduction générale

Dans un monde de plus en plus connecté, la collecte et l'analyse d'informations via des sondages en ligne sont devenues des pratiques courantes et indispensables pour les entreprises, les institutions académiques et les organisations de toutes tailles. Que ce soit pour évaluer la satisfaction client, mener des études de marché, recueillir des avis ou réaliser des enquêtes internes, les plateformes de sondage offrent un moyen efficace et rapide d'interagir avec un public ciblé. Cependant, de nombreuses solutions existantes peuvent s'avérer coûteuses, rigides ou manquer de fonctionnalités avancées adaptées aux besoins spécifiques des utilisateurs.

Le projet **SondagePro** s'inscrit dans cette dynamique en proposant une application web dynamique et personnalisable, conçue pour répondre à un large éventail de besoins en matière de sondage. L'objectif principal est de fournir une plateforme qui non seulement simplifie le processus de création et de diffusion de sondages, mais qui offre également des outils d'analyse puissants pour transformer les données brutes en informations exploitables. En tirant parti des technologies modernes de développement web, nous visons à créer une solution à la fois performante, sécurisée et facile à utiliser.

Ce rapport a pour but de documenter l'intégralité du processus de développement de SondagePro. Il couvre la justification du projet, les objectifs fixés, la méthodologie adoptée, les choix techniques, la conception détaillée de l'architecture et de la base de données, les étapes de développement, les fonctionnalités clés implémentées, les défis surmontés, ainsi que les perspectives d'évolution future. Il s'agit d'une synthèse académique et technique, reflétant l'approche rigoureuse et les apprentissages tirés de ce projet de fin d'études. L'ensemble du document est structuré pour offrir une compréhension claire et exhaustive de l'application SondagePro et de son processus de création.

---

## 1. Présentation du projet

### Contexte

Le besoin de collecter des données et des opinions de manière structurée est omniprésent dans de nombreux domaines. Les méthodes traditionnelles de sondage, souvent manuelles ou basées sur des outils limités, peuvent être chronophages, coûteuses et peu efficaces pour atteindre un large public. L'émergence des technologies web a ouvert la voie à des solutions plus agiles et interactives. Cependant, il existe un écart entre les outils génériques disponibles et les exigences spécifiques des projets nécessitant une grande flexibilité, une personnalisation poussée et une analyse de données sophistiquée. SondagePro vise à combler ce vide en offrant une plateforme conçue spécifiquement pour des sondages dynamiques et complexes.

### Objectifs

Les objectifs principaux du projet SondagePro sont les suivants :

- **Développer une plateforme de sondage complète** : Permettre aux utilisateurs de créer, gérer, diffuser et analyser des sondages de manière intuitive.
- **Offrir une personnalisation avancée** : Inclure des types de questions variés, des questions conditionnelles et la possibilité de créer des modèles réutilisables.
- **Assurer une analyse de données efficace** : Fournir des outils de visualisation graphique des résultats (via Chart.js) et des options d'exportation (CSV/Excel).
- **Garantir la sécurité et la confidentialité** : Mettre en œuvre un système d'authentification robuste, la gestion des permissions, la protection CSRF et la possibilité de sondages anonymes.
- **Proposer une interface utilisateur moderne et réactive** : Utiliser Bootstrap 5 pour une expérience utilisateur optimale sur différents appareils.
- **Faciliter la gestion des utilisateurs** : Permettre l'inscription, la connexion et la gestion des profils pour les créateurs de sondages et les participants.
- **Intégrer des notifications** : Informer les utilisateurs des événements importants liés à leurs sondages.

## Problématique

Comment concevoir et développer une application web de sondages dynamiques, basée sur le framework Django, qui soit à la fois performante, sécurisée, hautement personnalisable et capable de gérer efficacement la collecte, l'analyse et la présentation des données, tout en offrant une expérience utilisateur fluide et intuitive ? La problématique réside dans la conciliation des exigences de flexibilité fonctionnelle, de robustesse technique et d'ergonomie, en tenant compte des contraintes de temps et de ressources inhérentes à un projet universitaire.

## Solution proposée

La solution proposée est une application web modulaire, **SondagePro**, développée avec **Django** pour le backend, tirant parti de son architecture MVT (Model-View-Template) pour une séparation claire des préoccupations. La base de données **MySQL** est choisie pour sa fiabilité et sa performance. Le frontend utilise **HTML5**, **CSS3 (avec Bootstrap 5)** pour le style et la réactivité, et **JavaScript (ES6)** pour l'interactivité, notamment avec l'intégration de **Chart.js** pour les visualisations. L'application mettra en œuvre un système d'authentification et de gestion des permissions de Django, ainsi que des mécanismes de sécurité comme la protection CSRF. Des fonctionnalités clés telles que la création de sondages avec questions conditionnelles, la diffusion par lien, l'analyse graphique et l'exportation des résultats seront au cœur de la solution, garantissant une plateforme complète et efficace pour la gestion des sondages dynamiques.

---

## 2. Cahier des charges

Le cahier des charges de SondagePro a été élaboré pour définir précisément les attentes et les exigences du projet, tant sur le plan fonctionnel que non fonctionnel. Il sert de référence

tout au long du cycle de développement, assurant que le produit final répond aux besoins identifiés et aux contraintes techniques.

### Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit fournir aux utilisateurs. Pour SondagePro, ils incluent :

- **Gestion des utilisateurs :**
  - Inscription et connexion sécurisées des utilisateurs (créateurs de sondages et participants).
  - Gestion des profils utilisateurs (modification des informations personnelles, mot de passe).
  - Gestion des rôles et permissions (distinction entre créateurs et participants).
- **Création et gestion de sondages :**
  - Interface intuitive pour la création de nouveaux sondages.
  - Possibilité de définir le titre, la description, la date de début et de fin du sondage.
  - Ajout de différents types de questions (choix multiple, réponse unique, texte libre, échelle de Likert).
  - Gestion des questions conditionnelles (affichage d'une question en fonction de la réponse à une question précédente).
  - Configuration des sondages anonymes ou nominatifs.
  - Protection des sondages par mot de passe.
  - Activation/désactivation et suppression de sondages.
  - Création et utilisation de modèles de sondages réutilisables.
- **Diffusion de sondages :**
  - Génération de liens uniques pour le partage des sondages.
  - Possibilité de partager les sondages via différentes plateformes (e-mail, réseaux sociaux).
- **Participation aux sondages :**
  - Interface claire et simple pour les participants afin de répondre aux questions.
  - Validation des réponses pour assurer la cohérence des données.
  - Possibilité de modifier les réponses avant la soumission finale (si autorisé par le créateur).
- **Analyse et visualisation des résultats :**
  - Tableau de bord pour les créateurs de sondages affichant un aperçu des résultats.
  - Visualisation graphique des réponses pour chaque question (diagrammes circulaires, barres, etc. via Chart.js).
  - Statistiques détaillées (nombre de participants, taux de complétion, moyennes, etc.).

- **Exportation des données :**
  - Exportation des résultats des sondages au format CSV et Excel.
- **Notifications :**
  - Système de notification pour informer les créateurs de nouvelles participations ou de la fin d'un sondage.
  - Notifications pour les participants concernant les sondages auxquels ils sont invités.

### Besoins non fonctionnels

Les besoins non fonctionnels décrivent les qualités du système, telles que la performance, la sécurité et l'utilisabilité.

- **Performance :**
  - Temps de chargement des pages rapide, même avec un grand nombre de questions ou de participants.
  - Traitement efficace des soumissions de sondages.
  - Gestion optimisée des requêtes de base de données.
- **Sécurité :**
  - Protection contre les attaques CSRF (Cross-Site Request Forgery) et XSS (Cross-Site Scripting).
  - Authentification et autorisation robustes.
  - Sécurisation des données sensibles des utilisateurs et des réponses aux sondages.
  - Protection par mot de passe pour l'accès aux sondages.
  - Limitation par IP pour éviter les réponses multiples frauduleuses.
- **Utilisabilité :**
  - Interface utilisateur intuitive et facile à prendre en main pour tous les types d'utilisateurs.
  - Design réactif (Responsive Design) pour une accessibilité optimale sur différents appareils (ordinateurs, tablettes, smartphones).
  - Messages d'erreur clairs et informatifs.
- **Fiabilité :**
  - Disponibilité élevée du système.
  - Gestion des erreurs et récupération en cas de panne.
- **Maintenabilité et Évolutivité :**
  - Code bien structuré, commenté et conforme aux bonnes pratiques de développement Django.
  - Facilité d'ajout de nouvelles fonctionnalités et de modification des existantes.
  - Capacité à gérer une augmentation du nombre d'utilisateurs et de sondages.



## Contraintes techniques

Les contraintes techniques sont les limitations imposées par l'environnement ou les choix technologiques.

- **Framework** : Utilisation exclusive de Django pour le développement backend.
- **Langage de programmation** : Python pour le backend, JavaScript (ES6) pour le frontend.
- **Base de données** : MySQL comme système de gestion de base de données relationnelle.
- **Frontend** : Utilisation de Bootstrap 5 pour le framework CSS et Chart.js pour la visualisation des données.
- **Architecture** : Respect de l'architecture MVT (Model-View-Template) de Django.
- **Déploiement** : Le rapport ne couvre pas le déploiement mais doit prendre en compte les bonnes pratiques pour un déploiement futur.

## Technologies choisies

Les technologies sélectionnées pour le projet SondagePro sont les suivantes, justifiées par leur robustesse, leur popularité, leur écosystème riche et leur adéquation avec les objectifs du projet :

- **Python** : Langage de programmation principal pour le backend, apprécié pour sa lisibilité, sa polyvalence et sa vaste bibliothèque standard.
  - **Django** : Framework web de haut niveau en Python, qui encourage le développement rapide et la conception pragmatique et propre. Son architecture MVT est idéale pour des applications complexes.
  - **MySQL** : Système de gestion de base de données relationnelle open-source, reconnu pour sa performance, sa fiabilité et sa scalabilité, adapté aux besoins de stockage de données structurées.
  - **Django Forms** : Module de Django pour la création et la gestion de formulaires web, facilitant la validation des données et la protection CSRF.
  - **Django Authentication** : Système d'authentification et d'autorisation intégré à Django, offrant une gestion sécurisée des utilisateurs et des sessions.
  - **JavaScript (ES6)** : Langage de script côté client pour l'interactivité des pages web et les requêtes asynchrones (AJAX/Fetch API).
  - **Bootstrap 5** : Framework CSS populaire pour le développement d'interfaces web réactives et mobiles-first, assurant une présentation cohérente et moderne.
  - **Chart.js** : Bibliothèque JavaScript pour la création de graphiques simples et flexibles, essentielle pour la visualisation des résultats de sondages.
  - **Django REST Framework (optionnel)** : Cadre puissant et flexible pour la construction d'APIs web, envisagé pour des extensions futures comme une API mobile.
-

### 3. Analyse et conception

Cette section détaille l'approche adoptée pour l'analyse des besoins et la conception architecturale de l'application SondagePro. Une attention particulière a été portée à la modularité, à la scalabilité et à la maintenabilité du système, en s'appuyant sur les principes de conception logicielle et les spécificités du framework Django.

#### Architecture MVT Django

L'application SondagePro est conçue autour de l'architecture **Model-View-Template (MVT)** de Django, qui est une variation du modèle Model-View-Controller (MVC). Cette architecture favorise une séparation claire des préoccupations, ce qui facilite le développement, la maintenance et l'évolutivité du projet.

- **Modèle (Model)** : Représente la structure des données de l'application et gère l'interaction avec la base de données. Dans SondagePro, les modèles définissent les entités telles que Utilisateur, Sondage, Question, Réponse, etc., et encapsulent la logique métier liée aux données. Django ORM (Object-Relational Mapper) est utilisé pour interagir avec la base de données MySQL, permettant de manipuler les données Python sans écrire de requêtes SQL brutes.
- **Vue (View)** : Gère la logique métier et le traitement des requêtes HTTP. Une vue reçoit une requête web, interagit avec le modèle pour récupérer ou modifier des données, puis passe ces données à un template pour le rendu. Dans SondagePro, les vues sont responsables de la création de sondages, de l'affichage des questions, de la soumission des réponses, de l'authentification des utilisateurs et de la génération des statistiques.
- **Template (Template)** : Définit la structure de l'interface utilisateur. Les templates sont des fichiers HTML qui contiennent des balises et une logique de template Django pour afficher dynamiquement les données fournies par la vue. Ils sont responsables de la présentation visuelle de l'application, y compris les formulaires de sondage, les tableaux de bord des résultats et les pages d'authentification.

Cette architecture permet une organisation logique du code, où chaque composant a une responsabilité bien définie, ce qui améliore la clarté du projet et la collaboration entre développeurs.

## Description de la base de données

La base de données de SondagePro est conçue pour stocker toutes les informations relatives aux sondages, aux questions, aux choix de réponses et aux participations. MySQL a été choisi pour sa robustesse et sa performance. Les modèles Django sont directement mappés aux tables de la base de données, simplifiant ainsi les opérations CRUD.

Les principales entités de la base de données incluent :

User : Le modèle par défaut de Django qui gère l'authentification et les profils de tous les utilisateurs (Créateurs et Participants).

Poll : Contient les détails de chaque sondage (titre, description, dates de validité, mot de passe, créateur).

Question : Définit les questions associées à un sondage (texte, type de question, questions conditionnelles).

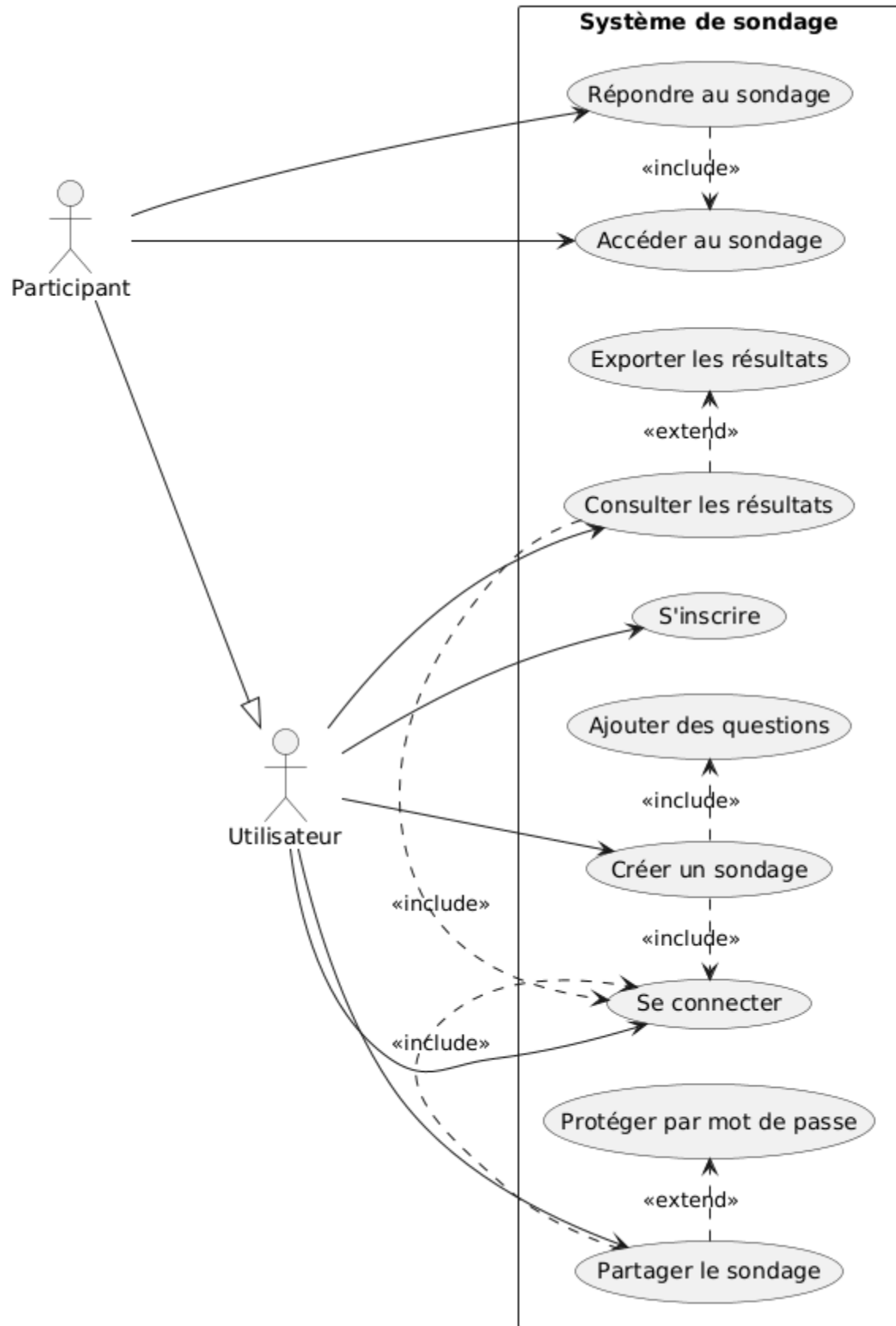
Choice : Stocke les options de réponse possibles pour les questions à choix multiples ou uniques.

Response : Enregistre la participation globale d'un utilisateur (ou d'une adresse IP anonyme) à un sondage spécifique.

Answer : Enregistre la réponse précise apportée par le participant à chaque question (texte ou choix sélectionné).

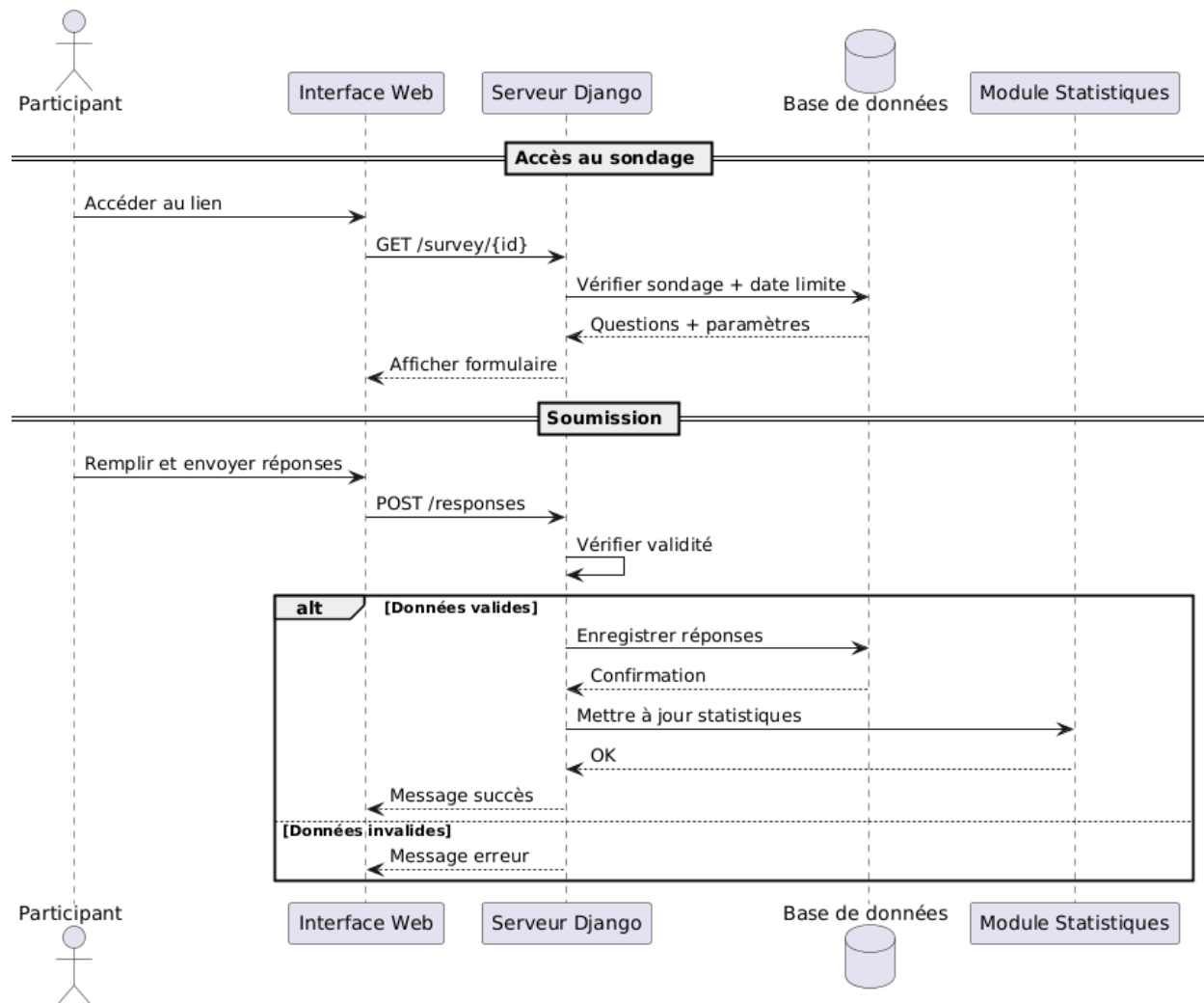
Notification : Gère les alertes envoyées aux créateurs lorsqu'une nouvelle participation est enregistrée.

## Diagramme de cas d'utilisation



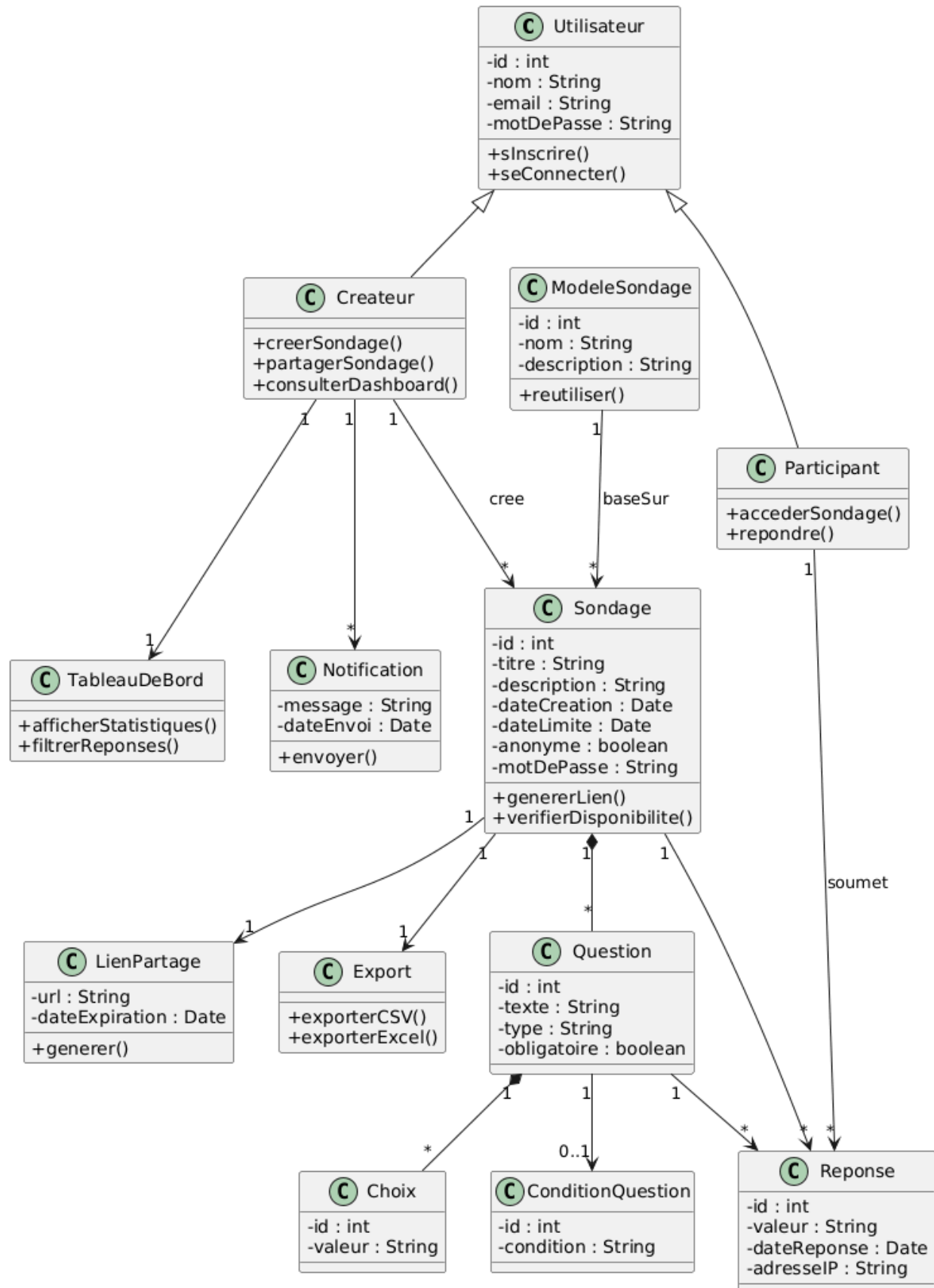
Le diagramme de cas d'utilisation identifie les principales fonctionnalités du système et les interactions entre les acteurs (utilisateurs) et le système. Pour SondagePro, les acteurs principaux sont le Créateur de Sondage et le Participant. Les cas d'utilisation incluent la création de sondages, la participation à un sondage, la visualisation des résultats, la gestion des utilisateurs, etc.

## Diagramme de séquence



Le diagramme de séquence illustre l'ordre chronologique des interactions entre les objets du système pour réaliser un cas d'utilisation spécifique. Par exemple, un diagramme de séquence pourrait montrer le flux d'événements lors de la création d'un sondage ou de la soumission d'une réponse.

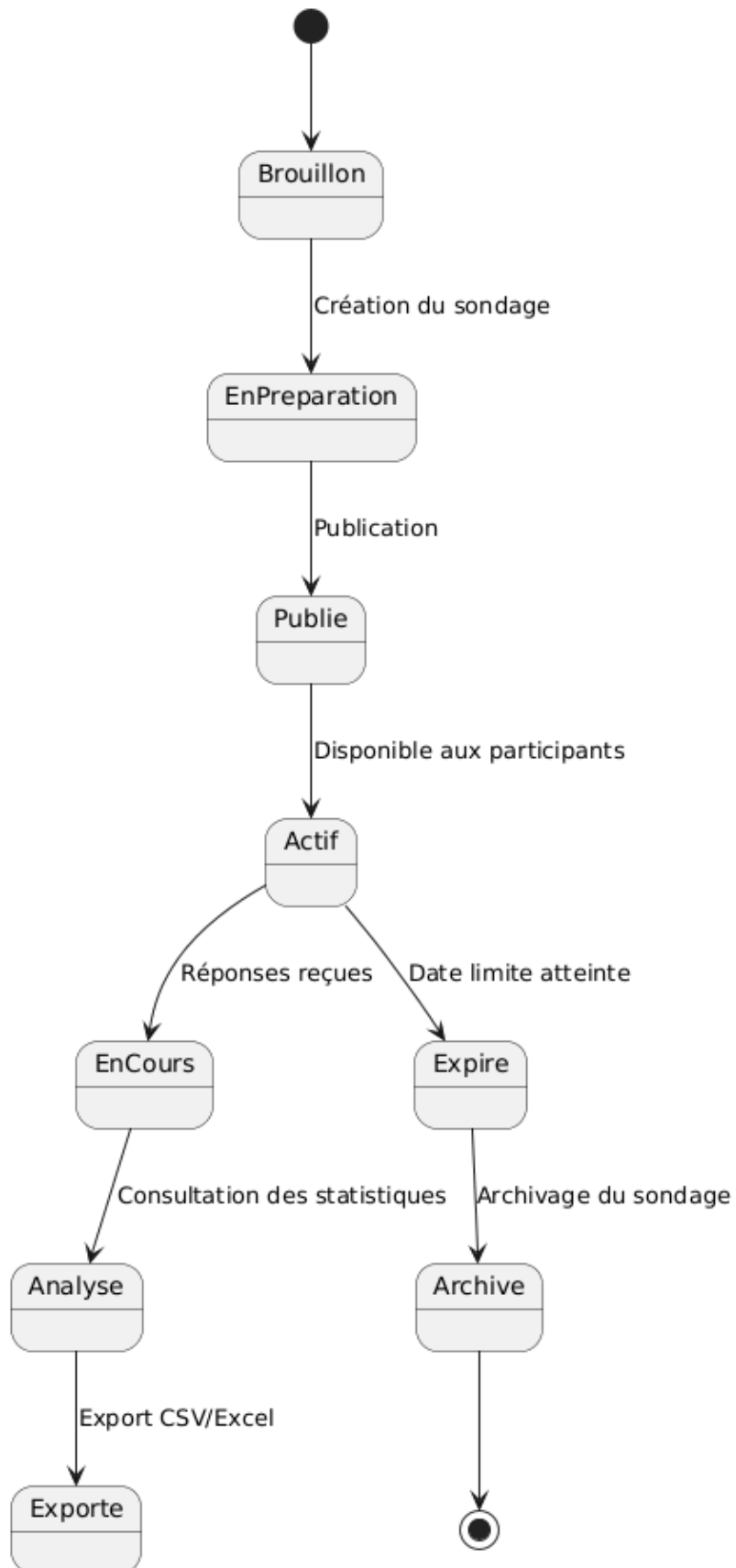
## Diagramme de classes



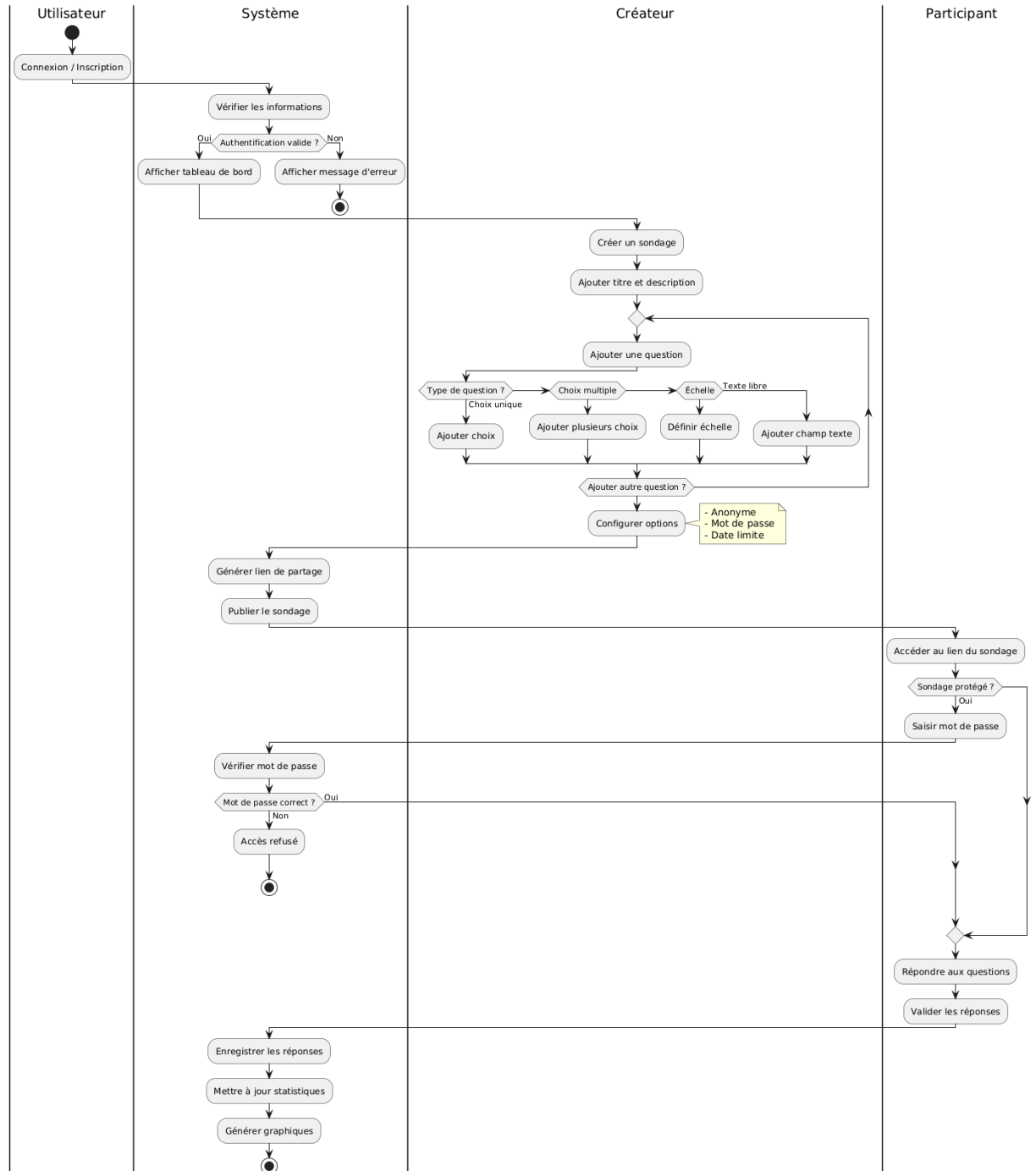
Le diagramme de classes représente la structure statique du système en montrant les classes, leurs attributs, leurs méthodes et les relations entre elles. Pour SondagePro, ce diagramme détaillera les classes correspondant aux modèles Django (Utilisateur, Sondage, Question, Réponse, etc.) et leurs associations.



## Diagramme de transition d'état



## Diagramme d'activité



Le diagramme d'activité modélise le flux de travail ou les processus métier. Il est utile pour décrire la logique derrière des opérations complexes, comme le processus de création d'un sondage avec des questions conditionnelles ou le flux de validation des réponses.

## 4. Conception de la base de données

La conception de la base de données est un élément crucial pour la robustesse et la performance de l'application SondagePro. Elle a été élaborée en tenant compte des besoins fonctionnels et non fonctionnels, en s'appuyant sur le modèle relationnel et en utilisant l'ORM de Django pour une intégration fluide avec le code Python. Les entités principales et leurs attributs sont décrits ci-dessous.

### Entités de la base de données

#### *User*

Cette entité représente les utilisateurs de la plateforme, qu'ils soient créateurs de sondages ou simples participants. Elle hérite du modèle `AbstractUser` de Django pour bénéficier des fonctionnalités d'authentification et d'autorisation intégrées.

- `id` (clé primaire)
- `username`, `email`, `password`, `first_name`, `last_name`, `is_staff`, `is_superuser`, `date_joined`

#### *Poll*

- `id` (clé primaire)
- `creator` (clé étrangère vers `User`)
- `title` (titre du sondage)
- `description`
- `is_active` (booléen)
- `is_anonymous` (booléen, permet les réponses anonymes)
- `password` (mot de passe du sondage)
- `expires_at` (date limite pour répondre)
- `limit_per_ip` / `limit_per_user` (entiers)
- `is_template` (booléen, indique si le sondage est un modèle)
- `custom_css` (design personnalisé)

#### *Question*

- id (clé primaire)
- poll (clé étrangère vers Poll)
- text (texte de la question)
- question\_type (choix: 'text', 'choice\_single', 'choice\_multiple', 'scale')
- order (ordre d'affichage)
- is\_required (booléen)
- depends\_on\_question (clé étrangère vers Question, pour logique conditionnelle)
- depends\_on\_choice (clé étrangère vers Choice, pour logique conditionnelle)

### *Choice*

- id (clé primaire)
- question (clé étrangère vers Question)
- text (texte du choix)
- order (ordre d'affichage)

### *Response*

- id (clé primaire)
- poll (clé étrangère vers Poll)
- user (clé étrangère vers User, optionnel)
- ip\_address (adresse IP du participant)
- created\_at

### *Answer*

- id (clé primaire)
- response (clé étrangère vers Response)
- question (clé étrangère vers Question)
- text\_answer (réponse textuelle ou numérique)
- choices (relation Many-to-Many vers Choice)

### **Notification**

- id (clé primaire)
- user (clé étrangère vers User)
- message (texte de la notification)
- is\_read (booléen)
- created\_at

Cette structure de base de données, implémentée via l'ORM de Django, assure une gestion efficace et cohérente des données de l'application SondagePro, tout en offrant la flexibilité nécessaire pour les évolutions futures.

---

## **5. Développement de l'application**

Le développement de l'application SondagePro a suivi une approche structurée, tirant parti des fonctionnalités robustes de Django pour le backend et des technologies modernes du frontend pour une expérience utilisateur riche. Cette section détaille les aspects clés de l'implémentation.

### **Structure du projet Django**

Le projet Django est centralisé pour plus de simplicité, avec un répertoire principal de configuration (poll\_app) et une application métier unique (polls) qui regroupe toute la logique métier.

/python Me

```
|— manage.py
|— poll_app/ (Configuration globale)
|   |— settings.py
|   |— urls.py
|   └— wsgi.py
|— polls/ (Application métier)
|   |— models.py (Logique de la base de données)
|   |— views.py (Contrôleurs et logique métier)
|   |— urls.py (Routage spécifique)
|   |— forms.py (Formulaires)
|   |— context_processors.py (Données globales pour les templates)
|   └— utils.py (Fonctions utilitaires et statistiques)
|— static/ (Fichiers CSS, JS, images)
└— templates/
    |— base.html
    |— polls/ (Vues spécifiques aux sondages)
    └— registration/ (Vues d'authentification)
```

## Gestion des routes (URLs)

Django utilise un système de routage basé sur des expressions régulières pour mapper les URL aux vues correspondantes. Le fichier `sondagepro_project/urls.py` sert de routeur principal, incluant les URL des différentes applications. Chaque application (`users`, `surveys`, `results`, `notifications`) possède son propre fichier `urls.py` pour une meilleure modularité.

Exemple de `sondagepro_project/urls.py`:

```
# Fichier poll_app/urls.py (Routeur principal)

from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('polls.urls')), # Toutes les routes sont gérées par l'app 'polls'

]
```

## Gestion des vues

Les vues Django sont des fonctions ou des classes Python qui reçoivent une requête web et renvoient une réponse web. Elles sont le cœur de la logique métier de l'application. Pour SondagePro, les vues gèrent des opérations telles que la création, la modification et la suppression de sondages, l'affichage des formulaires de réponse, la soumission des réponses, et la présentation des résultats.

Exemple de vue pour la création d'un sondage (`surveys/views.py`):



```

@login_required

def create_poll(request):
    if request.method == 'POST':
        # Votre application reçoit les données en format JSON
        data = json.loads(request.body)
        poll = Poll.objects.create(
            creator=request.user,
            title=data.get('title'),
            description=data.get('description'),
            is_anonymous=data.get('is_anonymous', False),
            password=data.get('password')
        )
        # Création des questions et choix à partir du JSON...
        return JsonResponse({'status': 'success', 'poll_id': poll.id})
    return render(request, 'polls/create_poll.html')

```

## Modèles Django

Les modèles Django définissent la structure de la base de données et fournissent une API de haut niveau pour interagir avec elle. Chaque modèle est une classe Python qui hérite de `django.db.models.Model` et représente une table dans la base de données. Les champs du modèle correspondent aux colonnes de la table.

Exemple de modèle Survey (`poll/models.py`):

```

class Poll(models.Model):
    creator = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
    title = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    is_anonymous = models.BooleanField(default=False)
    password = models.CharField(max_length=128, blank=True, null=True)
    expires_at = models.DateTimeField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

```

## Authentication (Django Authentication)

Le système d'authentification intégré de Django est utilisé pour gérer l'inscription, la connexion, la déconnexion et la gestion des mots de passe des utilisateurs. Un modèle d'utilisateur personnalisé (`models.py` ou `User`) peut être étendu pour inclure des champs spécifiques au projet, comme `from django.contrib.auth.models import User`.

## Sécurité CSRF

Django intègre une protection robuste contre les attaques CSRF (Cross-Site Request Forgery) via le middleware `CsrfViewMiddleware` et le tag de template `{% csrf_token %}`. Tous les formulaires POST dans SondagePro incluent ce token pour garantir la sécurité des requêtes.

## Responsive Design Bootstrap

L'interface utilisateur de SondagePro est développée avec **Bootstrap 5**, un framework CSS populaire qui assure un design réactif et mobile-first. Cela garantit que l'application est entièrement fonctionnelle et esthétiquement agréable sur une large gamme d'appareils, des ordinateurs de bureau aux smartphones.

## JavaScript AJAX / Fetch API

Pour une expérience utilisateur dynamique et sans rechargement de page, JavaScript (ES6) est utilisé avec l'API Fetch pour effectuer des requêtes AJAX. Cela est particulièrement utile pour des fonctionnalités comme la mise à jour en temps réel des résultats de sondage, la gestion des questions conditionnelles sans soumettre le formulaire entier, ou la validation asynchrone des entrées.

## Intégration Chart.js

**Chart.js** est intégré au frontend pour la visualisation graphique des résultats de sondage. Les données sont récupérées via des requêtes AJAX depuis les vues Django et sont ensuite utilisées par Chart.js pour générer des diagrammes interactifs (barres, circulaires, lignes) qui offrent une compréhension rapide et intuitive des réponses collectées.

---

## 6. Fonctionnalités implémentées

L'application SondagePro intègre un ensemble de fonctionnalités clés, développées pour offrir une solution complète et flexible de gestion de sondages. Ces fonctionnalités répondent aux besoins identifiés dans le cahier des charges et exploitent les capacités des technologies choisies.

### Authentification et Gestion des Utilisateurs

Le système d'authentification de Django a été mis en œuvre pour gérer l'accès à la plateforme. Les utilisateurs peuvent s'inscrire, se connecter et gérer leur profil. Une distinction est faite entre les créateurs de sondages et les participants, avec des rôles et permissions adaptés. Les sessions utilisateur sont gérées de manière sécurisée, garantissant que seuls les utilisateurs autorisés peuvent accéder aux fonctionnalités spécifiques.

### Création de Sondages Dynamiques

Les créateurs disposent d'une interface intuitive pour concevoir des sondages. Ils peuvent définir le titre, la description, les dates de début et de fin, et choisir parmi plusieurs types de questions : texte libre, choix unique (radio), choix multiples (checkbox), et échelle de Likert. La flexibilité de cette fonctionnalité permet de construire des questionnaires adaptés à une grande variété d'enquêtes.

### Questions Conditionnelles

Une fonctionnalité avancée implémentée est la gestion des questions conditionnelles. Cela permet d'afficher ou de masquer certaines questions en fonction des réponses données aux questions précédentes. Cette logique dynamique rend les sondages plus pertinents pour les participants et améliore la qualité des données collectées en évitant les questions non applicables.

### Partage via Lien Unique

Chaque sondage créé génère un lien URL unique. Ce lien peut être facilement partagé avec les participants via e-mail, réseaux sociaux ou toute autre plateforme. Cette méthode de diffusion simplifie l'accès aux sondages et maximise la portée de la collecte de données.

### Exportation des Résultats (CSV)

Les créateurs de sondages peuvent exporter les données brutes des réponses au format CSV. Cette fonctionnalité est essentielle pour permettre une analyse approfondie des données à l'aide d'outils externes (tableurs, logiciels d'analyse statistique). L'exportation inclut toutes les réponses, associées aux questions et, si applicable, aux informations du participant.

### Tableau de Bord et Statistiques

Un tableau de bord personnalisé est disponible pour les créateurs, offrant un aperçu rapide de l'état de leurs sondages (actifs, terminés, nombre de participants). Pour chaque sondage, des statistiques agrégées sont présentées, telles que le nombre total de réponses, le taux de complétion, et la répartition des réponses pour chaque question.

### Visualisation Graphique avec Chart.js

L'intégration de Chart.js permet une visualisation graphique interactive des résultats des sondages. Des diagrammes à barres, circulaires et linéaires sont utilisés pour représenter les distributions de réponses, les tendances et les proportions. Cette présentation visuelle facilite l'interprétation rapide des données et la détection des insights clés.

### Notifications

Un système de notification informe les créateurs des événements importants, tels que de nouvelles participations à leurs sondages ou l'approche de la date de fin d'un sondage. Ces notifications aident les créateurs à rester informés et à gérer leurs sondages de manière proactive.

### Protection par Mot de Passe

Pour les sondages nécessitant une confidentialité accrue, une option de protection par mot de passe a été implémentée. Seuls les participants disposant du mot de passe correct peuvent accéder au sondage, ajoutant une couche de sécurité supplémentaire.

### Limitation par IP

Pour prévenir les réponses multiples et garantir l'intégrité des données, une fonctionnalité de limitation par adresse IP a été mise en place. Elle permet de restreindre le nombre de participations à un sondage depuis une même adresse IP sur une période donnée, réduisant ainsi le risque de manipulation des résultats.

---

## 7. Difficultés rencontrées et solutions

Le développement d'une application web complexe comme SondagePro s'accompagne inévitablement de défis techniques. Cette section détaille les principales difficultés rencontrées et les solutions mises en œuvre pour les surmonter.

## Compatibilité MySQL

**Difficulté :** Lors de l'intégration de MySQL avec Django, des problèmes de compatibilité ont été rencontrés, notamment avec certaines versions de pilotes ou des configurations spécifiques de la base de données qui ne s'alignaient pas parfaitement avec l'ORM de Django. Des erreurs liées aux types de champs ou aux encodages de caractères sont apparues.

**Solution :** La solution a impliqué une vérification minutieuse des versions de `mysqlclient` (le connecteur Python pour MySQL) et de MySQL Server. Il a été nécessaire de s'assurer que les versions étaient compatibles et de configurer correctement l'encodage `utf8mb4` dans MySQL pour supporter une gamme complète de caractères. L'utilisation de l'ORM de Django a ensuite été ajustée pour s'assurer que les types de champs définis dans les modèles correspondaient aux types de données MySQL appropriés.

## Communication Frontend/Backend (AJAX/Fetch API)

**Difficulté :** La mise en place d'une communication asynchrone fluide entre le frontend (JavaScript) et le backend (Django) pour des fonctionnalités dynamiques (comme les questions conditionnelles ou la mise à jour des graphiques) a posé des défis. La gestion des requêtes AJAX, des réponses JSON et des erreurs côté serveur a nécessité une attention particulière.

**Solution :** L'utilisation de l'API Fetch de JavaScript a été privilégiée pour les requêtes asynchrones, en combinaison avec le Django REST Framework (pour les endpoints API) ou des vues Django renvoyant des réponses JSON. Une gestion robuste des erreurs côté client et serveur a été implémentée, avec des messages d'erreur clairs et des mécanismes de fallback. La sérialisation et la désérialisation des données JSON ont été gérées avec `json.dumps` et `json.loads` en Python, et `JSON.parse` et `JSON.stringify` en JavaScript.

## Gestion des relations ORM (One-to-Many, Many-to-Many)

**Difficulté :** La modélisation de relations complexes entre les entités (par exemple, un sondage a plusieurs questions, une question a plusieurs choix, un participant a plusieurs réponses) et leur manipulation via l'ORM de Django a parfois été délicate. Des erreurs courantes incluaient des problèmes de requêtes N+1 ou une mauvaise utilisation des `related_name`.

**Solution :** Une conception rigoureuse des modèles Django a été effectuée, en définissant clairement les clés étrangères (`ForeignKey`) et les relations Many-to-Many (`ManyToManyField`). L'utilisation de `select_related` et `prefetch_related` dans les requêtes de l'ORM a permis d'optimiser les performances en réduisant le nombre de requêtes à la base de données. Des tests unitaires ont été écrits pour valider le bon fonctionnement des relations et des requêtes.

## Validation des formulaires dynamiques

**Difficulté :** La validation des formulaires, en particulier ceux générés dynamiquement (comme les formulaires de création de sondages avec des questions et choix multiples via

des formsets), a présenté des complexités. Assurer que toutes les données soumises étaient valides et cohérentes avant l'enregistrement en base de données a été un point critique.

**Solution :** Django Forms et les `inlineformset_factory` ont été utilisés pour gérer la validation côté serveur. Des méthodes de validation personnalisées ont été ajoutées aux formulaires et aux modèles pour des règles de validation spécifiques (par exemple, s'assurer que la date de fin est postérieure à la date de début). Côté client, des validations JavaScript ont été implémentées pour fournir un feedback immédiat à l'utilisateur et améliorer l'expérience.

### Sécurité (CSRF, XSS, Injection SQL)

**Difficulté :** La protection de l'application contre les vulnérabilités de sécurité courantes telles que les attaques CSRF, XSS et les injections SQL a été une préoccupation constante.

**Solution :** Django offre des mécanismes de sécurité intégrés qui ont été pleinement exploités. La protection CSRF est activée par défaut et le tag `{% csrf_token %}` a été systématiquement utilisé dans tous les formulaires. Django ORM protège automatiquement contre les injections SQL en échappant les entrées. Pour les attaques XSS, les templates de Django échappent automatiquement le contenu par défaut, et des mesures supplémentaires ont été prises pour désinfecter les entrées utilisateur lorsque nécessaire.

### Gestion des permissions et accès

**Difficulté :** Définir et appliquer des permissions granulaires pour les différents types d'utilisateurs (administrateurs, créateurs, participants) et pour l'accès aux ressources (sondages, questions, réponses) a été un défi pour assurer que chaque utilisateur n'accède qu'aux données et fonctionnalités autorisées.

**Solution :** Le système d'authentification et d'autorisation de Django a été étendu. Des groupes d'utilisateurs et des permissions personnalisées ont été créés. Des décorateurs (`@login_required`, `@permission_required`) et des mixins (`LoginRequiredMixin`, `PermissionRequiredMixin`) ont été utilisés dans les vues pour restreindre l'accès. Des requêtes de base de données ont été construites pour filtrer les objets en fonction des permissions de l'utilisateur connecté.

---

## 8. Tests et validation

La phase de tests et de validation est essentielle pour garantir la qualité, la fiabilité et la conformité de l'application SondagePro aux spécifications définies. Une approche méthodique a été adoptée, combinant différents types de tests pour couvrir l'ensemble des fonctionnalités et des exigences non fonctionnelles.

## Tests fonctionnels

Les tests fonctionnels ont été menés pour vérifier que chaque fonctionnalité de l'application se comporte comme attendu, conformément au cahier des charges. Cela inclut :

- **Tests de création et gestion de sondages** : Vérification de la création de sondages avec différents types de questions, l'ajout de questions conditionnelles, la modification et la suppression de sondages.
- **Tests de diffusion** : S'assurer que les liens de partage sont générés correctement et que les sondages sont accessibles via ces liens.
- **Tests de participation** : Validation du processus de réponse aux sondages, y compris la soumission de réponses pour différents types de questions, la gestion des sondages anonymes et protégés par mot de passe.
- **Tests d'analyse et d'exportation** : Vérification de l'affichage correct des statistiques et des graphiques, ainsi que de l'exportation des données au format CSV.
- **Tests d'authentification et d'autorisation** : S'assurer que les utilisateurs peuvent s'inscrire, se connecter, se déconnecter et que les permissions sont correctement appliquées (par exemple, seuls les créateurs peuvent créer des sondages).

Des scénarios de test ont été élaborés pour chaque fonctionnalité, couvrant les cas nominaux et les cas limites. Des outils de test automatisés de Django (Django's TestCase et Client) ont été utilisés pour simuler les interactions utilisateur et vérifier les réponses du système.

## Validation des formulaires

La validation des formulaires est cruciale pour maintenir l'intégrité des données. Des tests spécifiques ont été effectués pour :

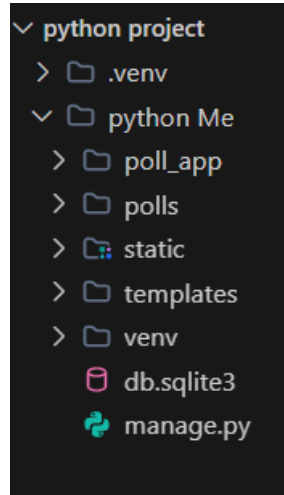
- **Validation côté client** : Vérification que les règles de validation JavaScript empêchent la soumission de données invalides avant l'envoi au serveur (par exemple, champs obligatoires, formats d'e-mail).
- **Validation côté serveur** : S'assurer que les validations définies dans les `forms.py` et `models.py` de Django fonctionnent correctement, rejetant les données incorrectes et affichant des messages d'erreur appropriés.
- **Tests de sécurité des formulaires** : Vérification de la présence et de l'efficacité du token CSRF pour toutes les soumissions de formulaires POST.

## Tests utilisateurs (User Acceptance Testing - UAT)

Bien que le projet soit universitaire, une phase de tests utilisateurs simplifiée a été envisagée pour évaluer l'expérience utilisateur et l'ergonomie de l'application. Des utilisateurs représentatifs ont été invités à interagir avec l'application pour identifier les éventuels problèmes d'utilisabilité, les bugs non détectés et les améliorations potentielles. Leurs retours ont été précieux pour affiner l'interface et le flux utilisateur.

## Captures des résultats

Plusieurs captures d'écran ont été réalisées tout au long du développement pour documenter l'interface utilisateur et les différentes fonctionnalités de l'application. Ces captures sont insérées dans le rapport pour illustrer les points clés et fournir une représentation visuelle du système.





```

└─ python project
  └─ > .venv
  └─ python Me
    └─ poll_app
      └─ __pycache__
        ├── __init__.py
        ├── asgi.py
        ├── settings.py
        ├── urls.py
        └── wsgi.py
      └─ polls
        ├── __pycache__
        ├── migrations
        ├── __init__.py
        ├── admin.py
        ├── apps.py
        ├── forms.py
        ├── models.py
        ├── tests.py
        ├── urls.py
        ├── utils.py
        └── views.py
      └─ static
        ├── css
        ├── js
        └── dummy.txt
      └─ templates
        ├── polls
        ├── registration
        └── base.html
      └─ venv
        ├── db.sqlite3
        └── manage.py

```

---

## 9. Perspectives d'amélioration

Le projet SondagePro, bien que fonctionnel et robuste, offre de nombreuses pistes d'amélioration et d'extension pour l'avenir. Ces perspectives visent à enrichir l'expérience utilisateur, à augmenter les capacités d'analyse et à étendre la portée de l'application.

### Notifications temps réel

Actuellement, les notifications sont basées sur un système de polling ou de déclenchement différé. L'intégration de technologies de notifications en temps réel, telles que WebSockets (par exemple, avec Django Channels), permettrait aux créateurs de sondages de recevoir instantanément des alertes concernant de nouvelles participations ou des commentaires, améliorant ainsi la réactivité de la plateforme.

### API mobile

Le développement d'une API RESTful dédiée, en utilisant Django REST Framework, ouvrirait la voie à la création d'applications mobiles natives (iOS et Android) pour SondagePro. Cela permettrait aux utilisateurs de créer, gérer et participer à des sondages directement depuis leurs appareils mobiles, offrant une flexibilité accrue et une meilleure accessibilité.

### IA pour analyse des réponses

L'intégration de capacités d'intelligence artificielle et de traitement du langage naturel (NLP) pourrait révolutionner l'analyse des réponses textuelles. Par exemple, l'IA pourrait être utilisée pour :

- **Analyse de sentiment** : Évaluer le sentiment général exprimé dans les réponses ouvertes.
- **Extraction de mots-clés** : Identifier les thèmes récurrents et les concepts importants.
- **Regroupement de réponses** : Catégoriser automatiquement les réponses similaires pour faciliter l'analyse qualitative.

### Système de recommandation

Un système de recommandation pourrait suggérer aux créateurs de sondages des types de questions, des structures de sondages ou des modèles basés sur leurs préférences passées ou sur la popularité de certains formats. Cela accélérerait le processus de création et améliorerait la qualité des sondages.

### Multilingue

L'internationalisation (i18n) et la localisation (l10n) de l'application permettraient à SondagePro de supporter plusieurs langues. Cela rendrait la plateforme accessible à un public mondial et faciliterait la création de sondages dans différentes langues.

## Hébergement cloud

Actuellement, le projet est développé dans un environnement local. Une perspective majeure serait le déploiement de l'application sur une plateforme cloud (comme AWS, Google Cloud, Azure ou Heroku). Cela garantirait une haute disponibilité, une scalabilité automatique et une meilleure performance pour un nombre croissant d'utilisateurs et de sondages.

---

## 10. Conclusion générale

Le projet **SondagePro – Application Web de Sondages Dynamiques** a été une entreprise ambitieuse et enrichissante, aboutissant à la création d'une plateforme web fonctionnelle et robuste pour la gestion de sondages en ligne. En s'appuyant sur le framework Django et un ensemble de technologies modernes, nous avons réussi à concevoir une solution qui répond aux besoins complexes de collecte, d'analyse et de visualisation de données, tout en offrant une expérience utilisateur intuitive et sécurisée.

Ce rapport a détaillé l'ensemble du processus, depuis l'analyse des besoins et la conception architecturale basée sur l'architecture MVT de Django, jusqu'au développement des fonctionnalités clés telles que la création de sondages dynamiques, les questions conditionnelles, la gestion des utilisateurs, l'exportation des résultats et la visualisation graphique avec Chart.js. Les défis techniques rencontrés, notamment en matière de compatibilité MySQL, de communication asynchrone et de gestion des relations ORM, ont été surmontés grâce à une approche méthodique et à l'exploitation des meilleures pratiques de développement.

SondagePro démontre la puissance et la flexibilité de Django pour le développement rapide d'applications web complexes. Il constitue une base solide pour de futures évolutions, avec des perspectives prometteuses telles que l'intégration de notifications en temps réel, le développement d'une API mobile, l'application de l'intelligence artificielle pour l'analyse des réponses, et le déploiement sur des infrastructures cloud. Ce projet a non seulement permis de mettre en pratique des connaissances techniques approfondies, mais a également souligné l'importance d'une planification rigoureuse, d'une conception soignée et d'une résolution créative des problèmes dans le cycle de vie du développement logiciel.

En somme, SondagePro est plus qu'une simple application de sondage ; c'est une preuve de concept d'une solution complète et évolutive, prête à s'adapter aux exigences futures du monde numérique.

---

## 11. Webographie

Cette section liste les principales ressources et documentations techniques qui ont été consultées et utilisées tout au long du développement du projet SondagePro.

- **Documentation officielle de Django** : La ressource primordiale pour comprendre le framework, son architecture MVT, l'ORM, le système d'authentification et les meilleures pratiques de développement. <https://docs.djangoproject.com/>
  - **Documentation de Bootstrap** : Guide complet pour l'utilisation du framework CSS, essentiel pour le responsive design et la mise en page de l'interface utilisateur. <https://getbootstrap.com/docs/5.3/>
  - **Documentation de Chart.js** : Référence pour l'intégration et la personnalisation des graphiques interactifs pour la visualisation des résultats de sondages. <https://www.chartjs.org/docs/latest/>
  - **Documentation MySQL** : Informations sur la configuration de la base de données, les types de données et l'optimisation des requêtes. <https://dev.mysql.com/doc/>
  - **Stack Overflow** : Une plateforme communautaire indispensable pour la résolution de problèmes techniques spécifiques et l'obtention de solutions à des défis de programmation. <https://stackoverflow.com/>
  - **OpenAI Documentation** : Consultée pour des idées sur l'intégration future de l'IA et du NLP pour l'analyse des réponses. <https://platform.openai.com/docs/>
  - **MDN Web Docs** : Une ressource complète pour les technologies web côté client (HTML, CSS, JavaScript, Fetch API), essentielle pour le développement frontend. <https://developer.mozilla.org/fr/docs/Web>
- 

## 11. Annexes

Les annexes regroupent des éléments complémentaires qui, bien que non intégrés directement dans le corps principal du rapport, fournissent des informations détaillées et un support technique additionnel.

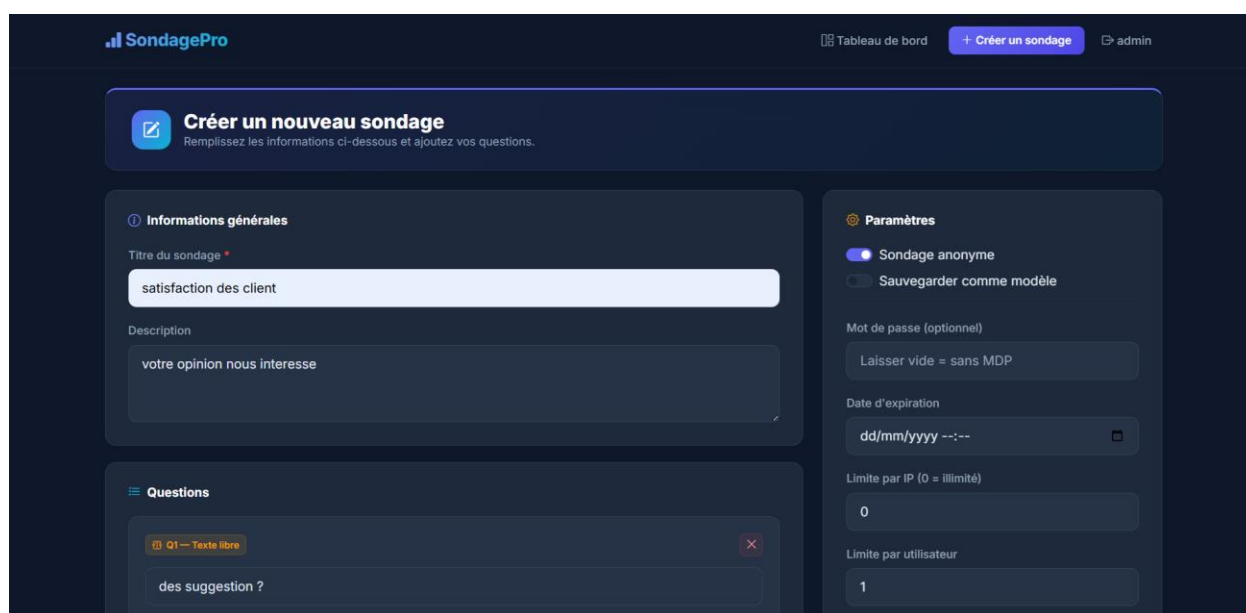
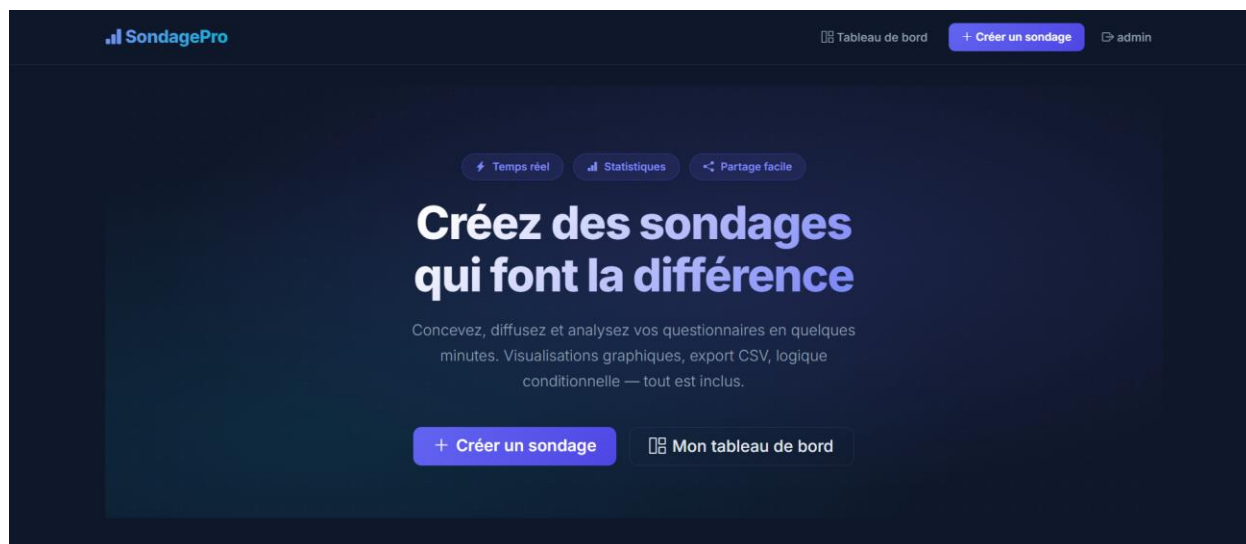
### Extraits de code

Cette section contiendrait des extraits de code significatifs illustrant des implémentations clés, des fonctions utilitaires complexes ou des configurations importantes. Par exemple :

- Modèle `Question` avec gestion des questions conditionnelles.
- Vue `create_survey` avec `formsets`.
- Code JavaScript pour l'intégration de `Chart.js`.

### Captures d'écran

Des captures d'écran supplémentaires de l'application en fonctionnement, montrant différentes interfaces utilisateur, formulaires, tableaux de bord et visualisations de résultats.



SondagePro

Tableau de bordCréer un sondageadmin

Questions

Q1 — Texte libre

des suggestion ?

Réponse obligatoire

Q2 — Choix multiple

votre opinion

Réponse obligatoire

Options de réponse :

satisfait

non satisfait

Ajouter une option

+ Texte libre

+ Choix unique

+ Choix multiple

+ Échelle (1-5)

Paramètres

Sondage anonyme

Sauvegarder comme modèle

Mot de passe (optionnel)

Laisser vide = sans MDP

Date d'expiration

dd/mm/yyyy --:--

Limite par IP (0 = illimité)

0

Limite par utilisateur

1

Publier le sondage

Le sondage sera immédiatement actif.

SondagePro

Tableau de bordCréer un sondageadmin

Bonjour, admin👋

Nouveau sondage

Gérez vos sondages et suivez vos statistiques.

Total sondages1

Actifs1

Réponses totales0

Mes sondages1 sondage

| TITRE                                                     | DATE DE CRÉATION | RÉPONSES | QUESTIONS | STATUT | ACTIONS              |
|-----------------------------------------------------------|------------------|----------|-----------|--------|----------------------|
| satisfaction des client<br>Anonyme ⌚ Expire le 30/05/2026 | 📅 13 Mai 2026    | 0        | 2         | Actif  | <div>📊🔗📄⏸️👁️🗑️</div> |

© 2026 SondagePro — Accueil • Tableau de bord

SondagePro

ConnexionInscription

Bon retour !

Connectez-vous à votre compte SondagePro.

Nom d'utilisateur

Mot de passe

Se connecter

Pas encore de compte ? Créer un compte

© 2026 SondagePro — Accueil • S'inscrire

SondagePro

ConnexionInscription

Créer un compte

Rejoignez SondagePro et commencez à créer vos sondages.

Nom d'utilisateur

Requis, 150 caractères maximum. Uniquement des lettres, nombres et les caractères « @ », « . », « + », « - » et « \_ ».

Adresse e-mail

votre@email.com

Mot de passe

Votre mot de passe ne peut pas trop ressembler à vos autres informations personnelles.

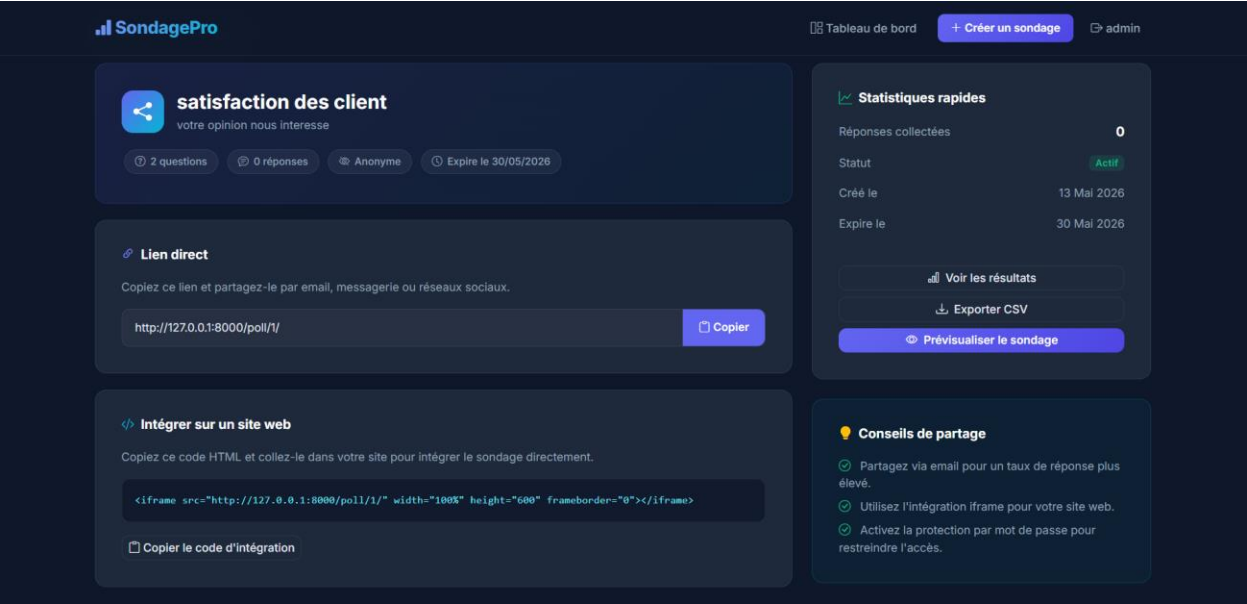
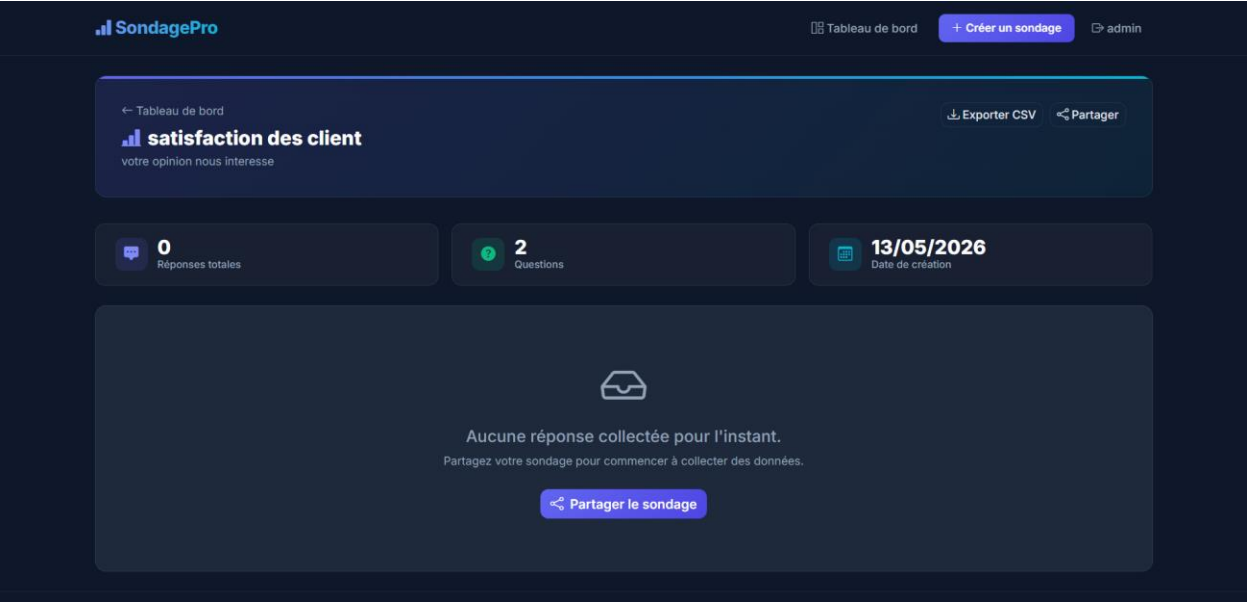
Votre mot de passe doit contenir au minimum 8 caractères.

Votre mot de passe ne peut pas être un mot de passe couramment utilisé.

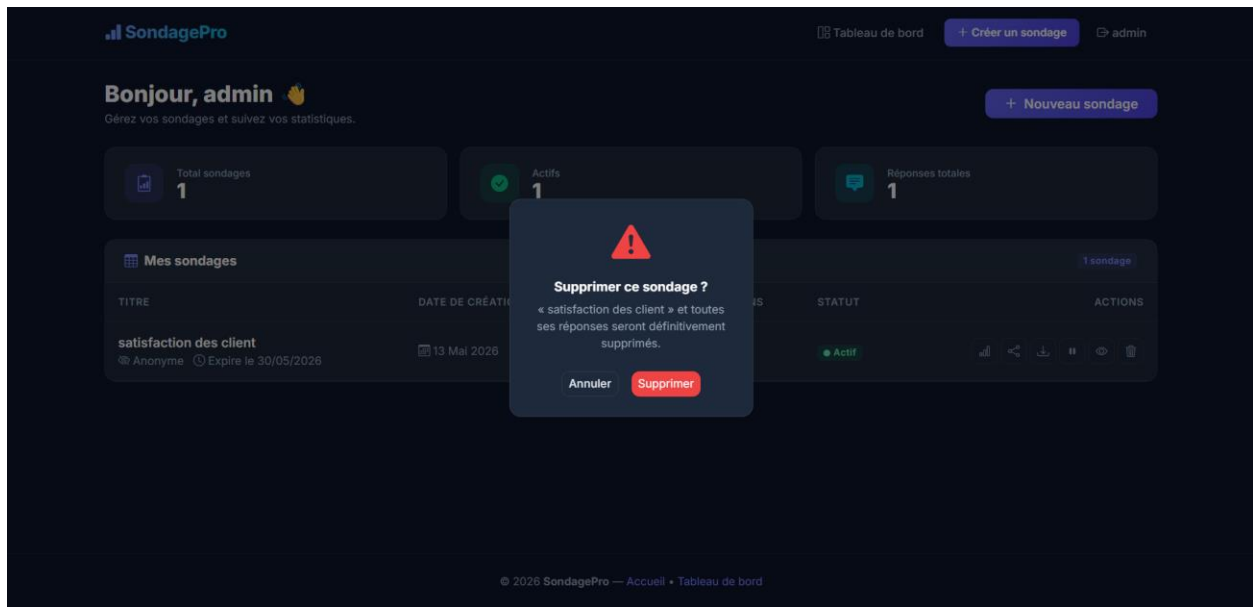
Votre mot de passe ne peut pas être entièrement numérique.

Confirmation du mot de passe

Saisissez le même mot de passe que précédemment, pour vérification.







## Structure du projet

Une arborescence détaillée du projet Django, incluant tous les fichiers et dossiers, pour donner une vue d'ensemble de l'organisation du code.

/python Me

```

├── manage.py          # Point d'entrée pour les commandes Django
├── poll_app/          # Dossier de configuration du projet
│   ├── settings.py    # Paramètres (BDD, Apps, Sécurité)
│   ├── urls.py        # Routeur principal
│   └── wsgi.py         # Interface serveur
├── polls/             # Application principale (Logique métier)
│   ├── models.py      # Définition des tables (Poll, Question, Answer...)
│   ├── views.py       # Logique des pages et traitement des données
│   ├── urls.py        # Routes spécifiques à l'application
│   ├── forms.py       # Formulaires de connexion et filtres
│   ├── utils.py        # Calcul des statistiques et export CSV
│   └── context_processors.py # Gestion des notifications globales
└── static/            # Fichiers CSS et JavaScript

```

```
| └─ css/          # Styles personnalisés (dashboard.css, home.css...)
|   └─ js/         # Scripts (create_poll.js, chart.js...)
└─ templates/     # Fichiers HTML
    └─ base.html   # Squelette commun (Navbar, Footer)
    └─ polls/      # Pages (home, dashboard, results...)
    └─ registration/ # Authentification (login, register)
    └─ db.sqlite3   # Base de données locale (pour développement)
```

### Commandes d'installation et de démarrage

Liste des commandes nécessaires pour configurer l'environnement de développement, installer les dépendances, migrer la base de données et démarrer l'application Django.

*# 1. Cloner Le dépôt*

```
git clone https://github.com/Mouad05-mz/project-python-sondage-app.git
cd "python Me"
```

*# 2. Créer l'environnement virtuel*

```
python -m venv venv
```

*# 3. Activer l'environnement virtuel (Windows)*

```
.\venv\Scripts\activate
```

*# 4. Installer les dépendances*

```
pip install -r requirements.txt
```

*# 5. Appliquer les migrations de la base de données*

```
python manage.py makemigrations
```

```
python manage.py migrate
```

*# 6. Créer un compte administrateur*

```
python manage.py createsuperuser
```

*# 7. Lancer le serveur*

*python manage.py runserver*

---